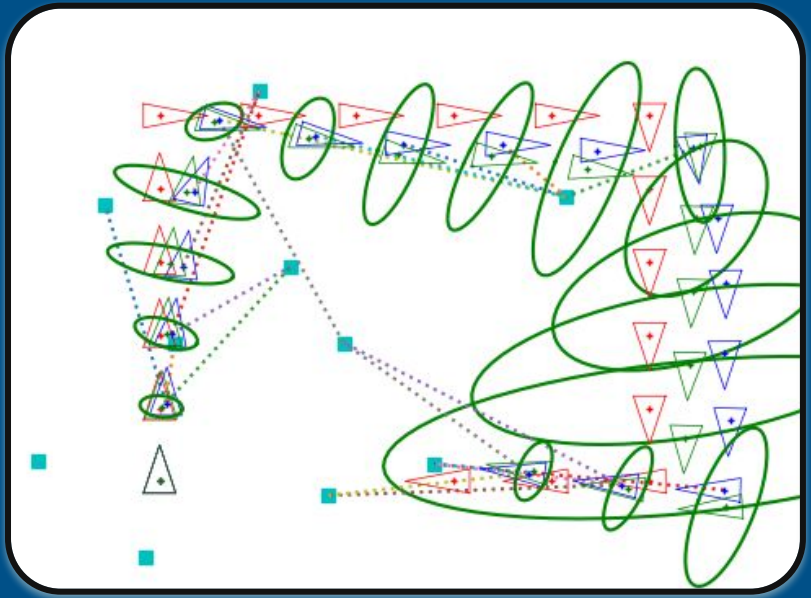
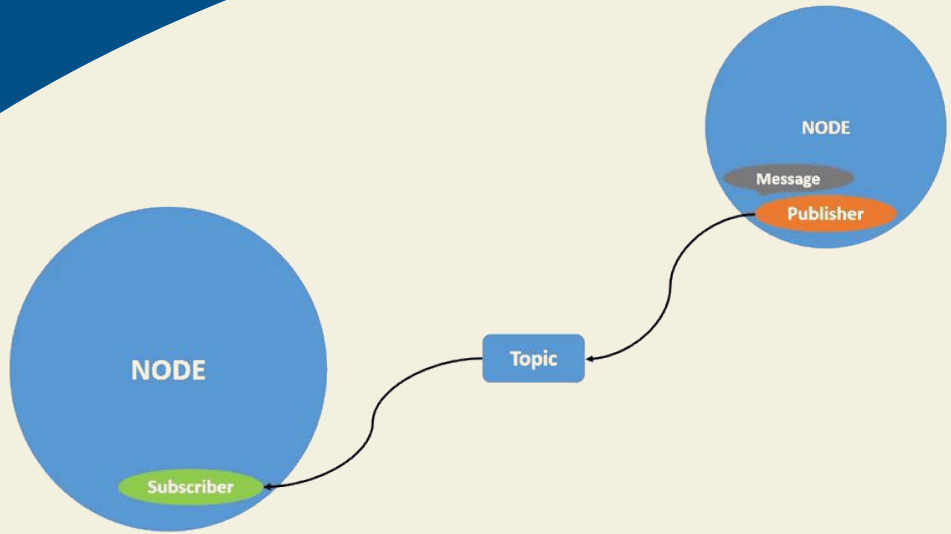
I402: Principios de la Robótica Autónoma

Clases Prácticas
Tutorial 2



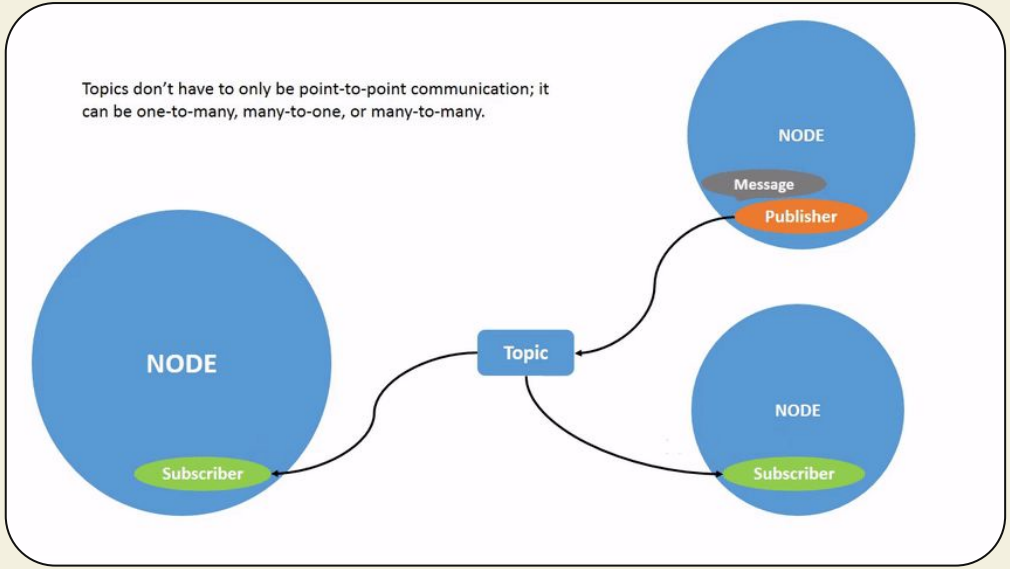
Universidad de SanAndrés

Nodos, topicos, subscribir y publicar

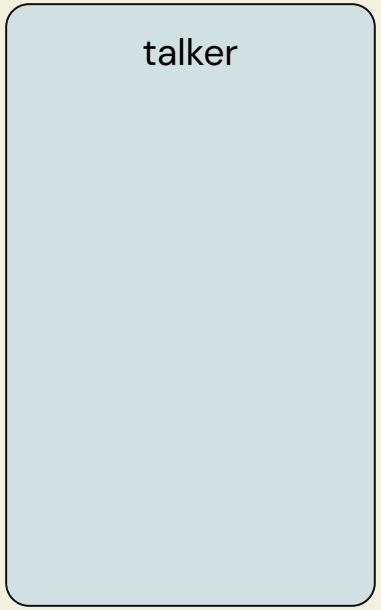


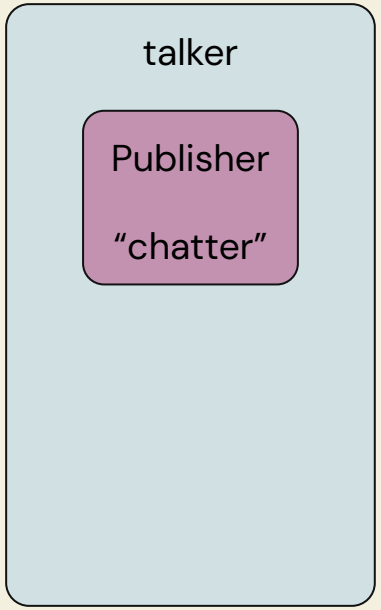
ROS: Arquitectura

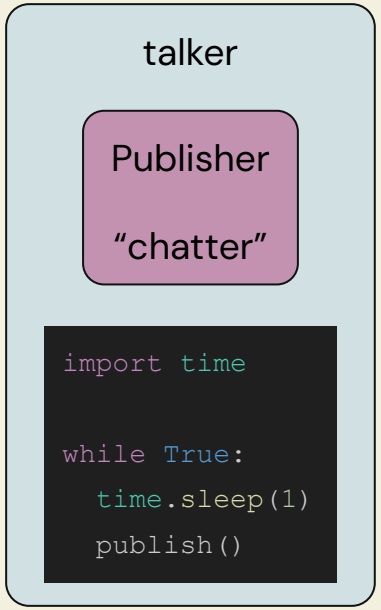
- Nodo
  - Corre procesos
  - Puede publicar y suscribirse a topics
- Topics
  - Canales para mandar mensajes
- Mensajes
  - Datos que los nodos usan para comunicarse

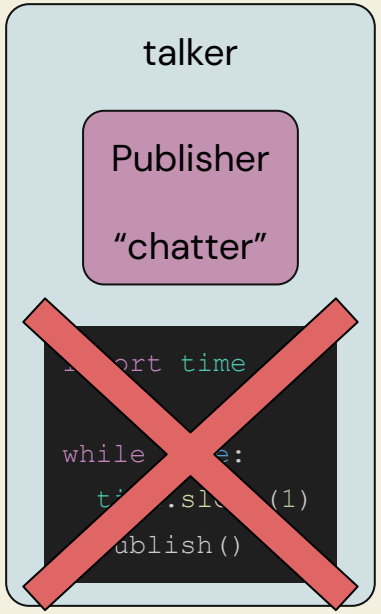


Nodo que contínuamente manda mensajes cada segundo

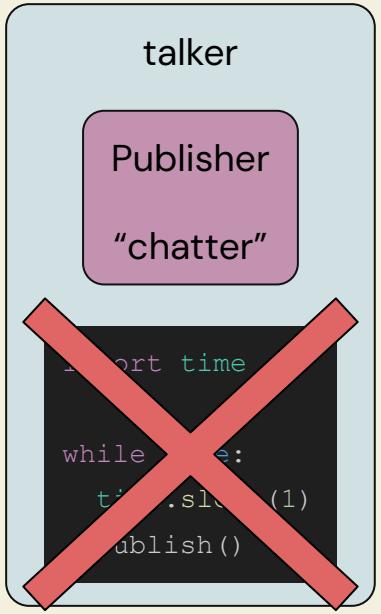



talker
Publisher
“chatter”

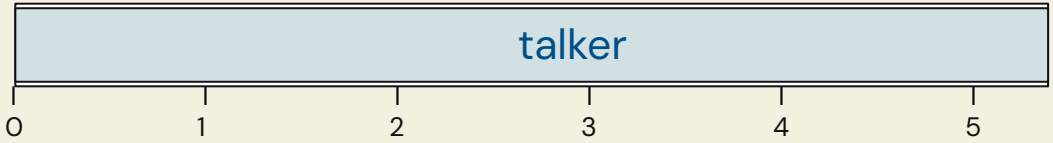

talker
Publisher
“chatter”
import time
while True:
time.sleep(1)
publish()

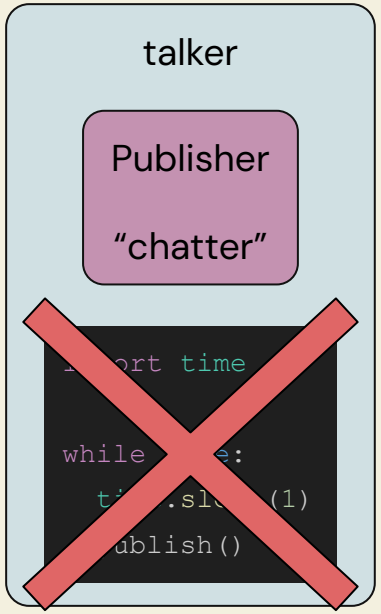


Desde el punto de vista de ROS este nodo está ocupado el 100% del tiempo. Si por alguna razón se supone que mi nodo tiene que tambien escuchar otros tópicos o correr otros procesos no lo va a poder hacer



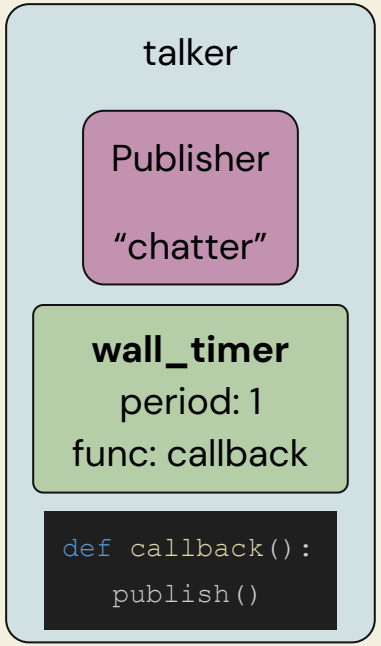
Comienzo de Ejecución

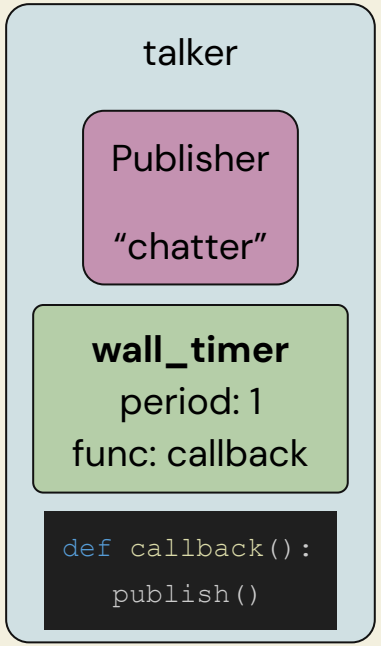




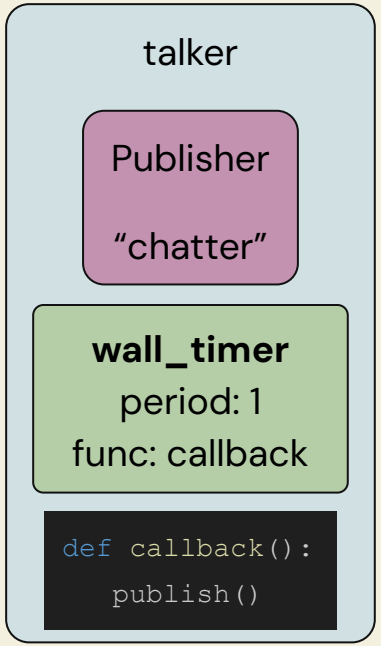
¿Qué quiere ROS?

ROS quiere ocupar el menor tiempo posible

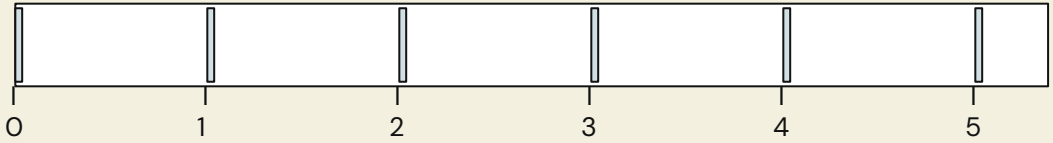

talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()

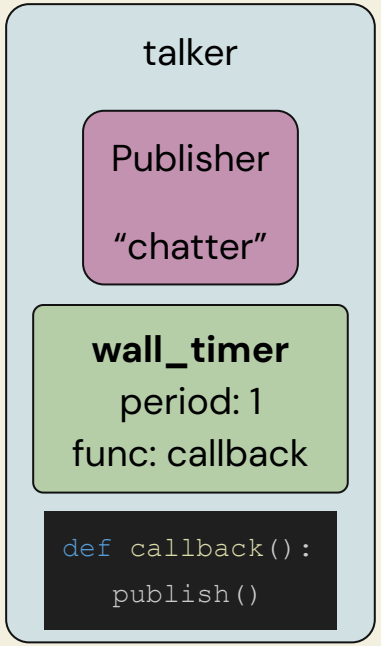


Ahora este nodo está ocupado solo unas milésimas de segundo cada segundo.

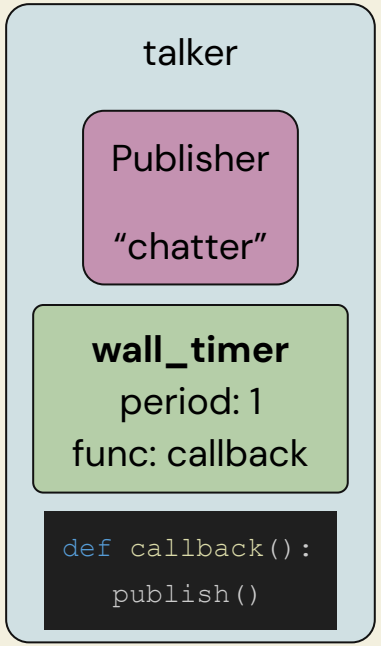


Comienzo de Ejecución

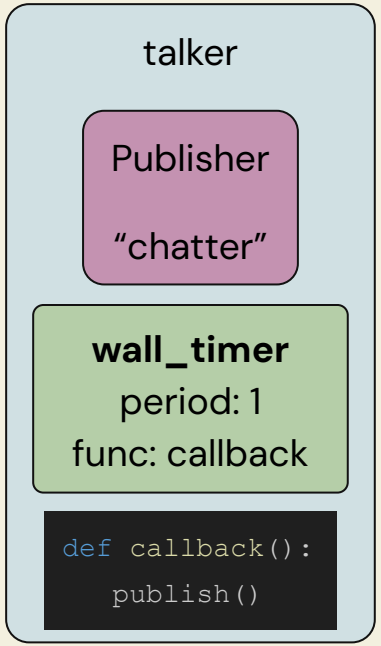




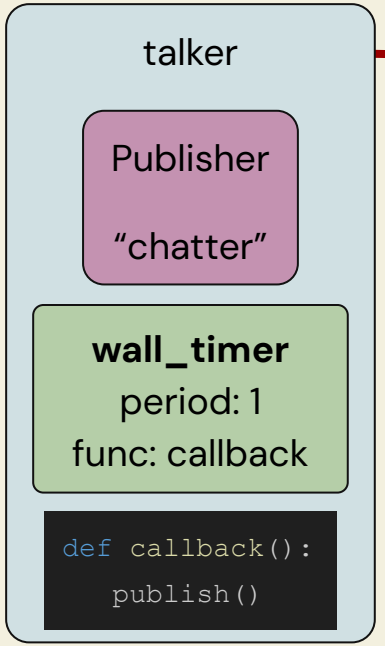

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



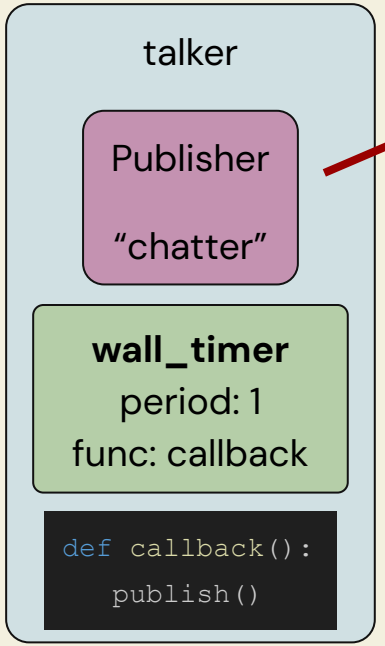

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



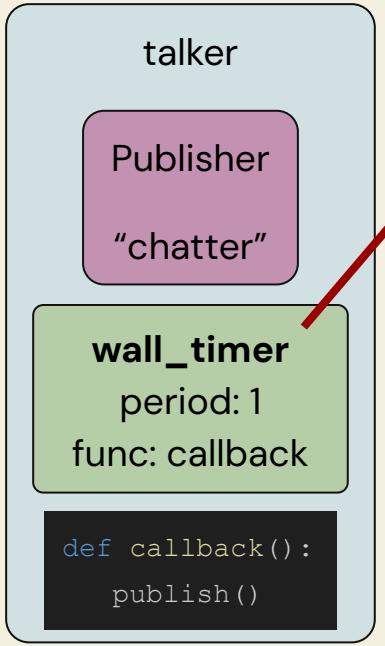

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



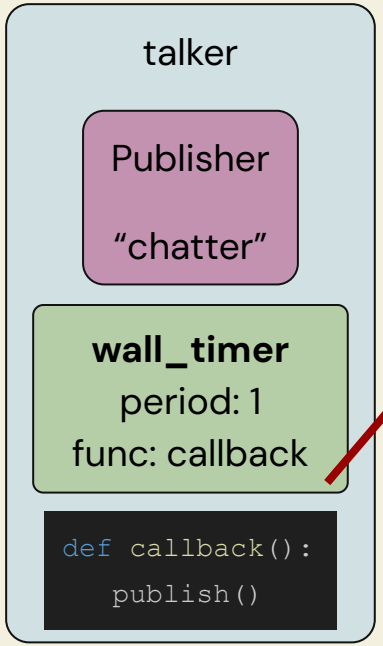

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



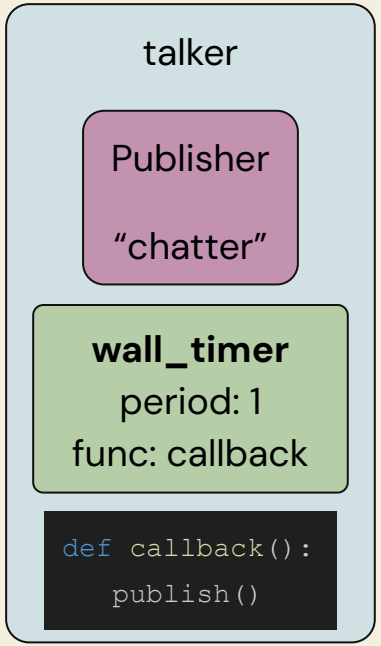

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```




```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



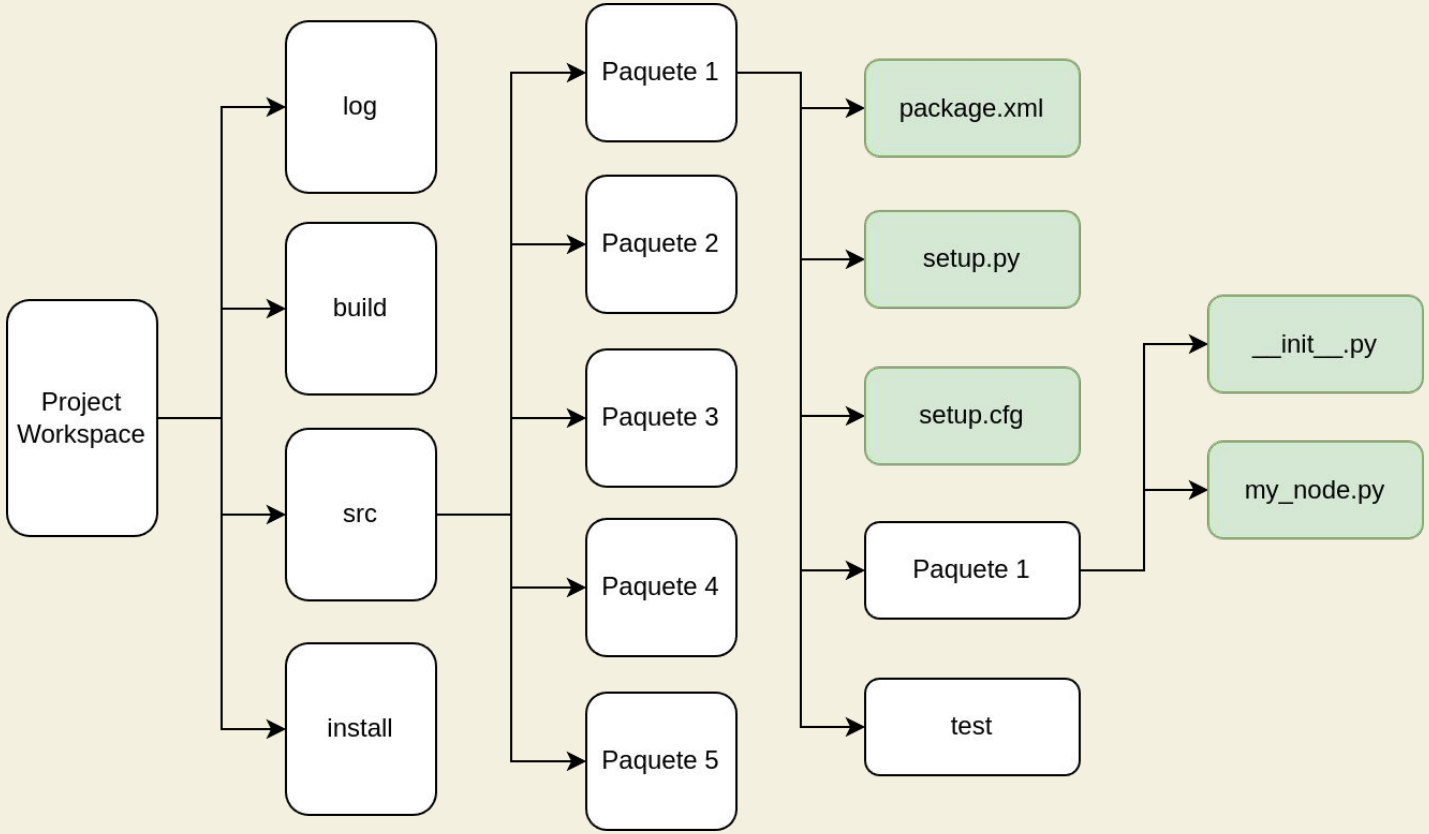

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic 'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world {self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```




```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1, self.timer_callback)  # 1s
        self.get_logger().info("I will publish to the topic
'chatter'")

    def timer_callback(self):
        msg = String()
        msg.data = f'hello world
{self.get_clock().now().to_msg().sec}'
        self.get_logger().info(msg.data)
        self.publisher.publish(msg)

def main():
    rclpy.init()
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

¿Ahora que tengo el código cómo lo corro con ROS?

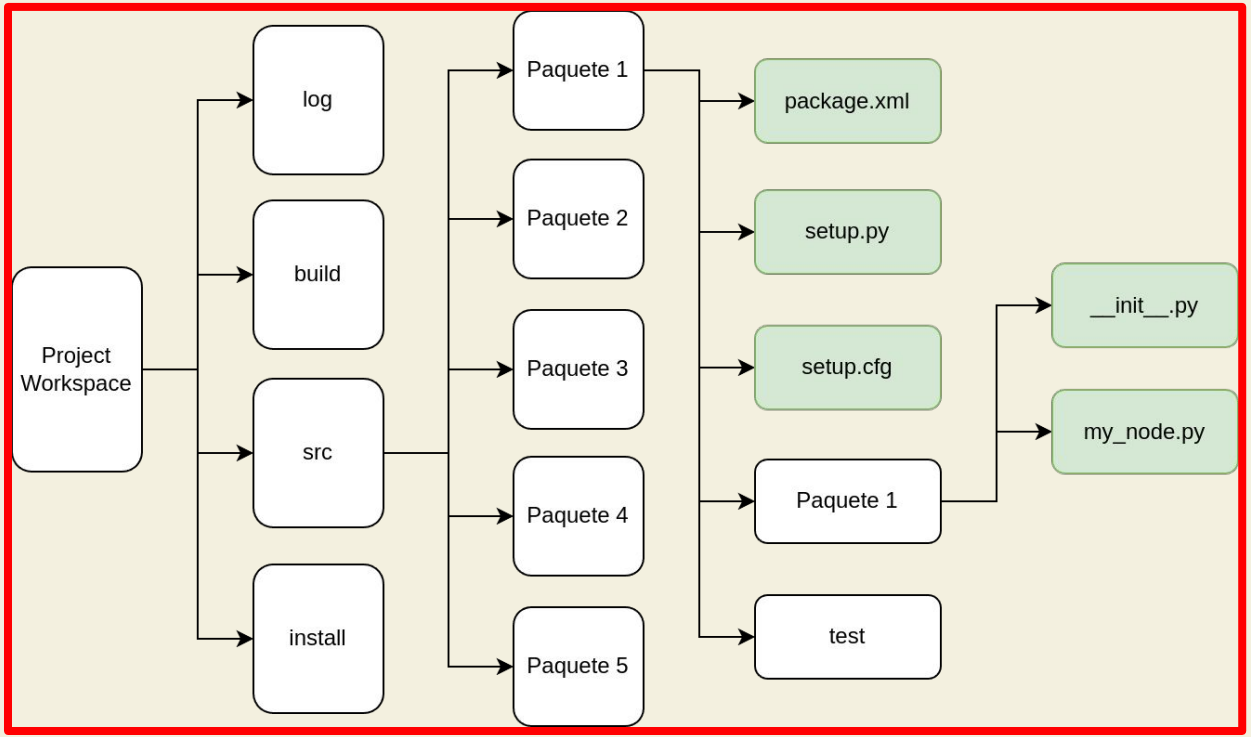
Estructura del WorkSpace



Comentario sobre uso de computadoras del laboratorio

Las computadoras son compartidas. Entonces, para evitar problemas de plagio:

1. No dejen su carpeta de workspace en la computadora
2. Cada vez que la usen descarguen o traigan de algun lado sus archivos

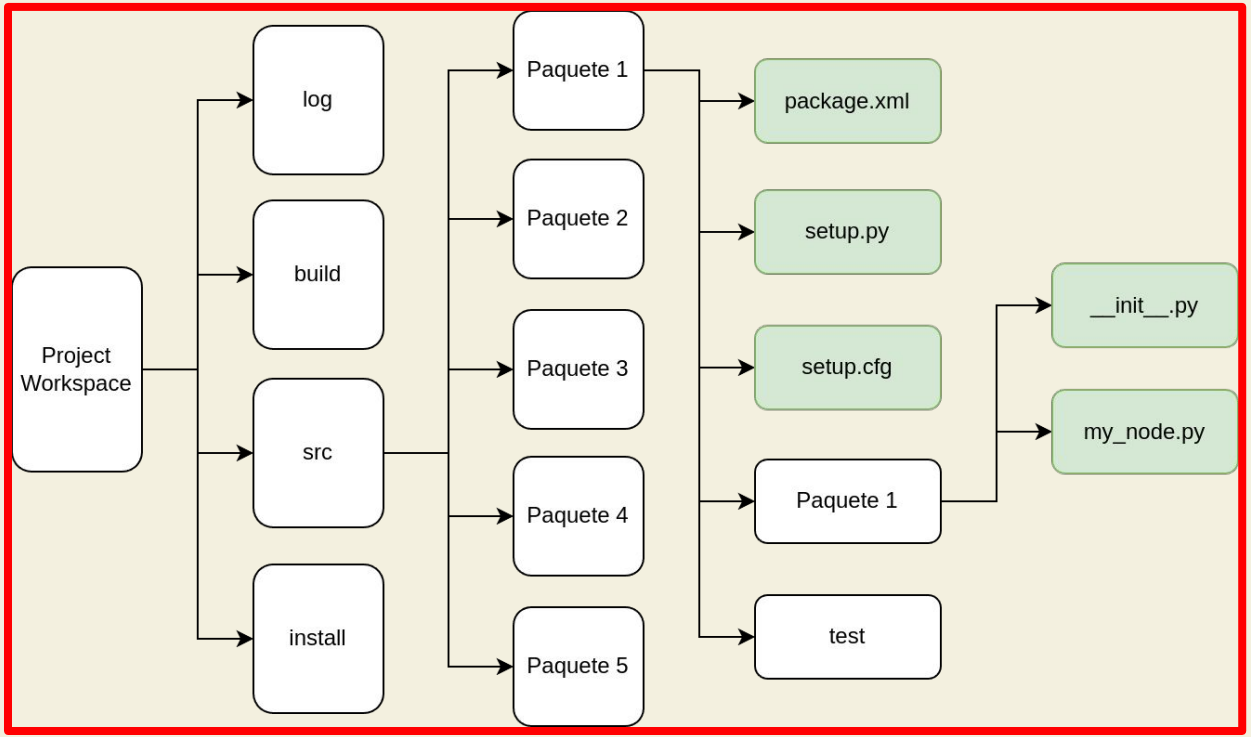


Comentario sobre uso de computadoras del laboratorio

Las computadoras son compartidas. Entonces, para evitar problemas de plagio:

3. BORREN LA CARPETA UNA VEZ QUE HAYAN TERMINADO

Solo guarden el ws con la carpeta src, las demás se generan automáticamente con los pasos a seguir



Nuestro paquete

1. Creamos el workspace y dentro creamos la carpeta src

```
mkdir -p ~/Documents/ros2_ws/src
```

2. Creamos el paquete dentro de src

```
ros2 pkg create --build-type ament_python --license Apache-2.0 <pkg_name>
```

3. Entramos en la carpeta de ros2_ws/src/pkg_name/pkg_name
4. Creamos y agregamos el código de nuestro nodo
5. Modificamos package.xml y setup.py

Nuestro paquete

5. Modificamos package.xml y setup.py

package.xml → Llenamos los primeros datos y luego agregamos lo siguiente después de la licencia

```
<exec_depend>rclpy</exec_depend>

<exec_depend>std_msgs</exec_depend>
```

setup.py → Llenamos los datos hasta la licencia y dentro de “console_scripts” agregamos el “entry_point”

```
entry_points={
        'console_scripts': [
            'talker = pkg_name.my_file:main',
        ],
},
```

Nuestro paquete

1. Creamos el workspace y dentro creamos la carpeta src

```
mkdir -p ~/Documents/ros2_ws/src
```

2. Creamos el paquete dentro de src

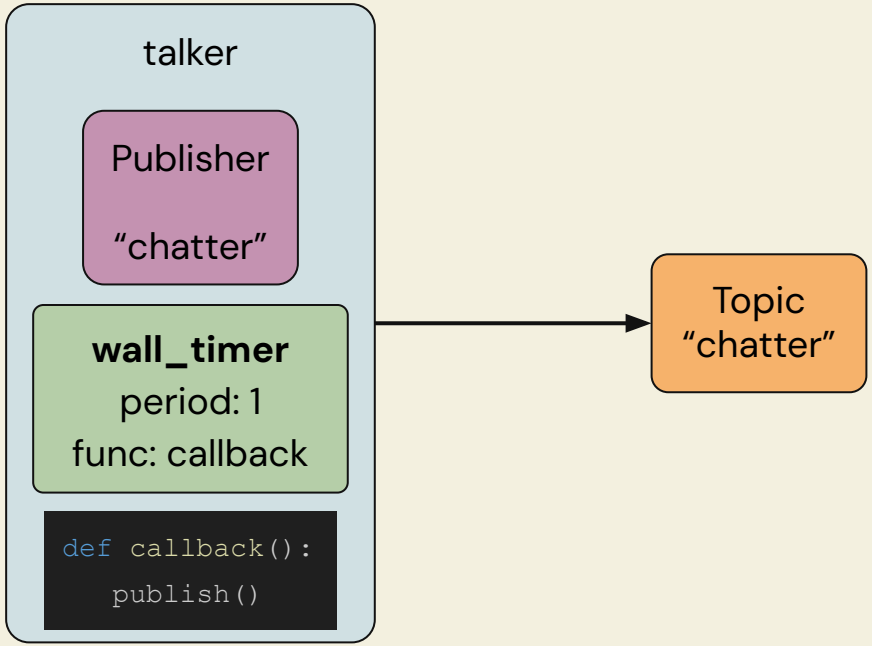
```
ros2 pkg create --build-type ament_python --license Apache-2.0 <pkg_name>
```

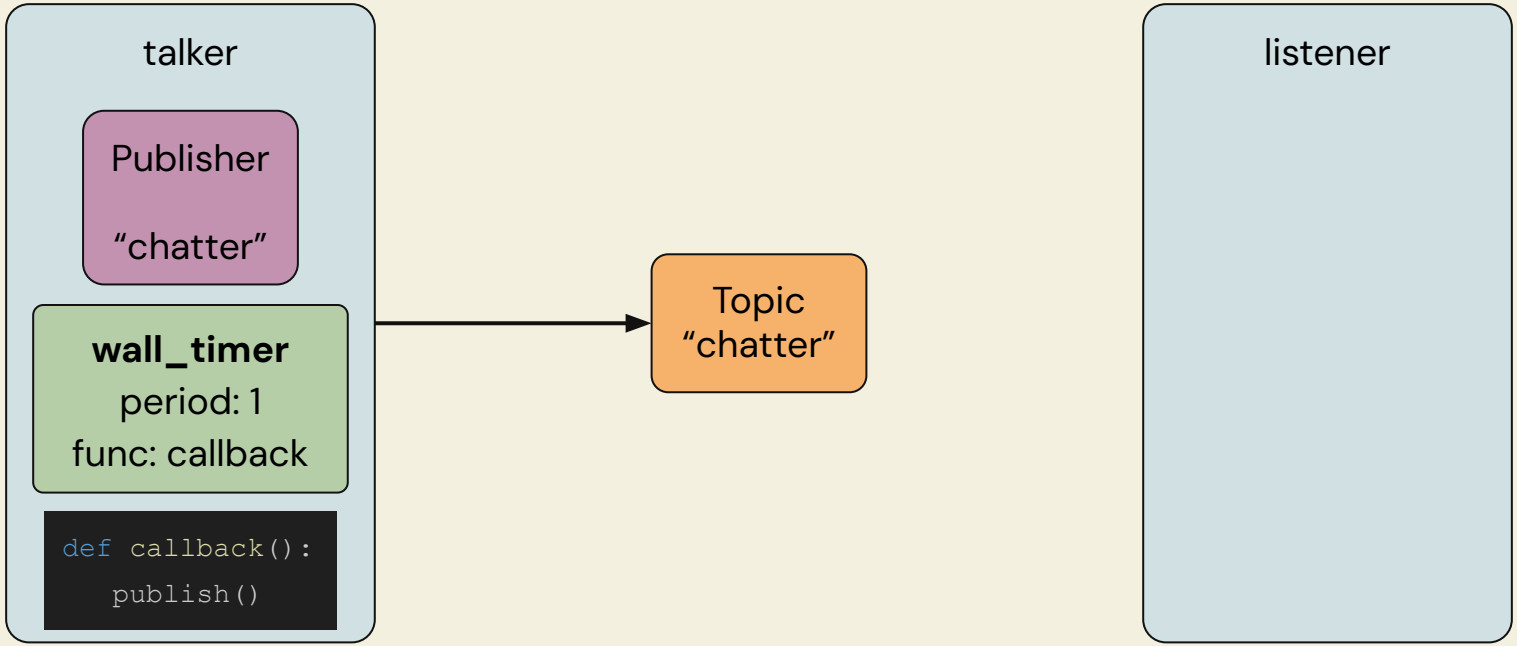
3. Entramos en la carpeta de ros2_ws/src/pkg_name/pkg_name
4. Creamos y agregamos el código de nuestro nodo
5. Modificamos package.xml y setup.py
6. Compilamos todo desde la carpeta del workspace

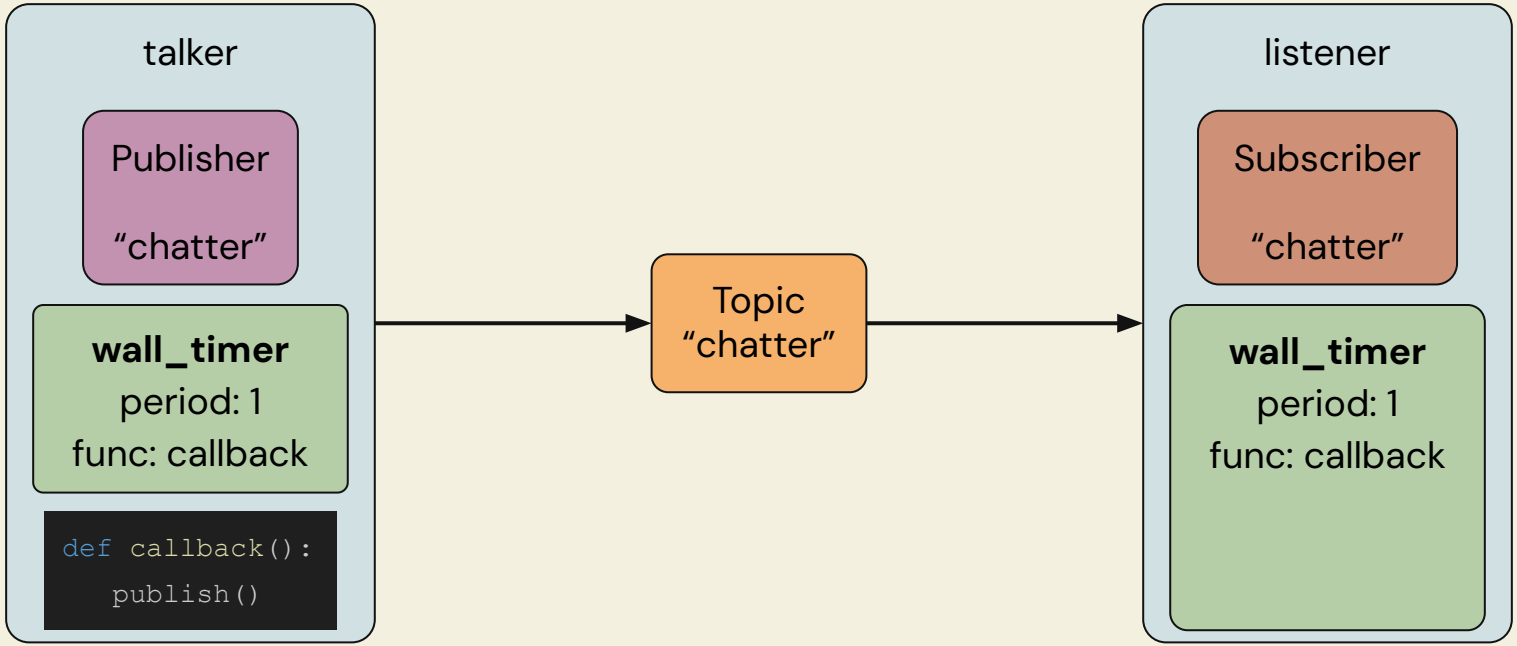
```
colcon build --symlink-install
```

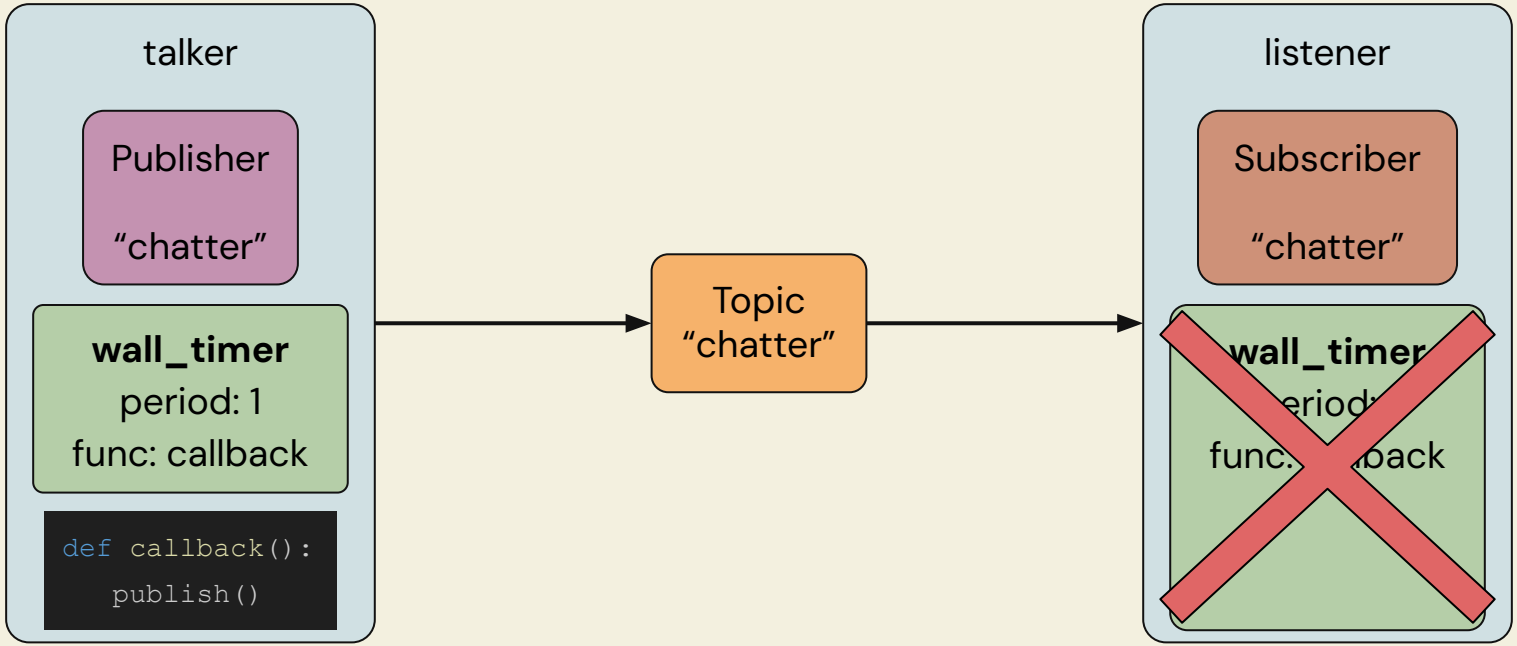
7. Hacemos “source install/setup.bash” (ojo si no tienen bash)
8. Corremos el nodo

```
ros2 run pkg_name talker
```

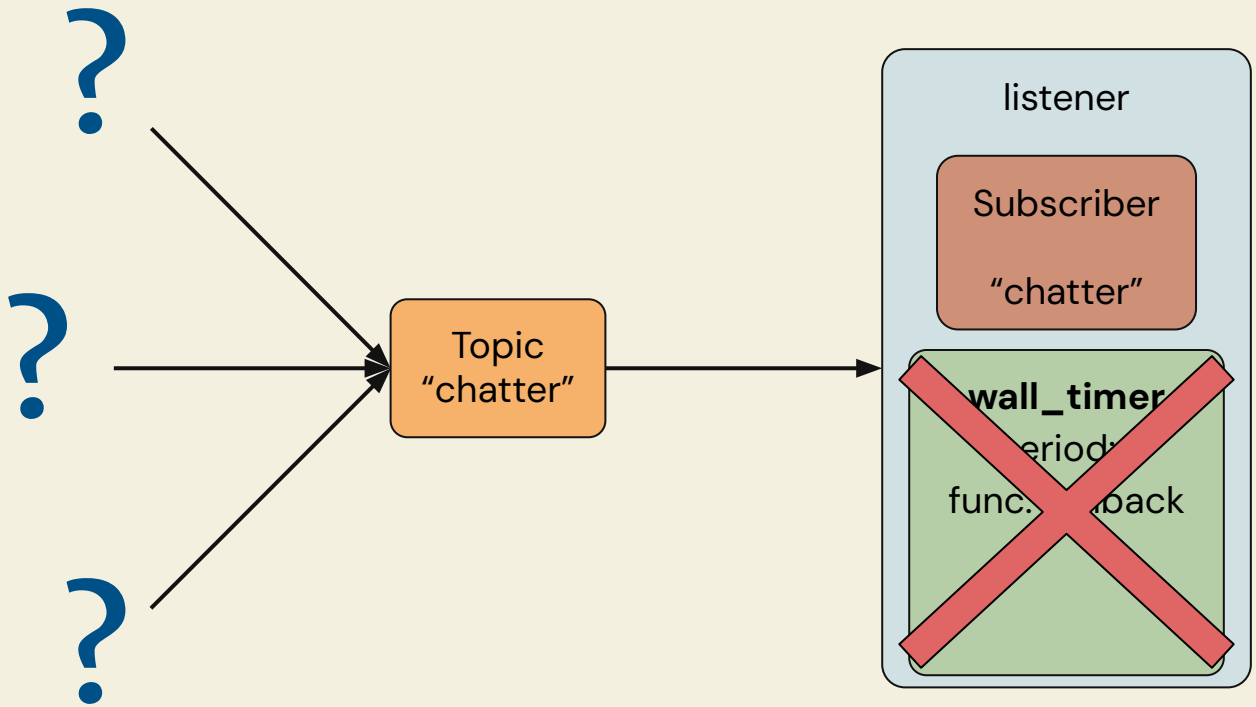

talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()
Topic
“chatter”


talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()
Topic
“chatter”
listener

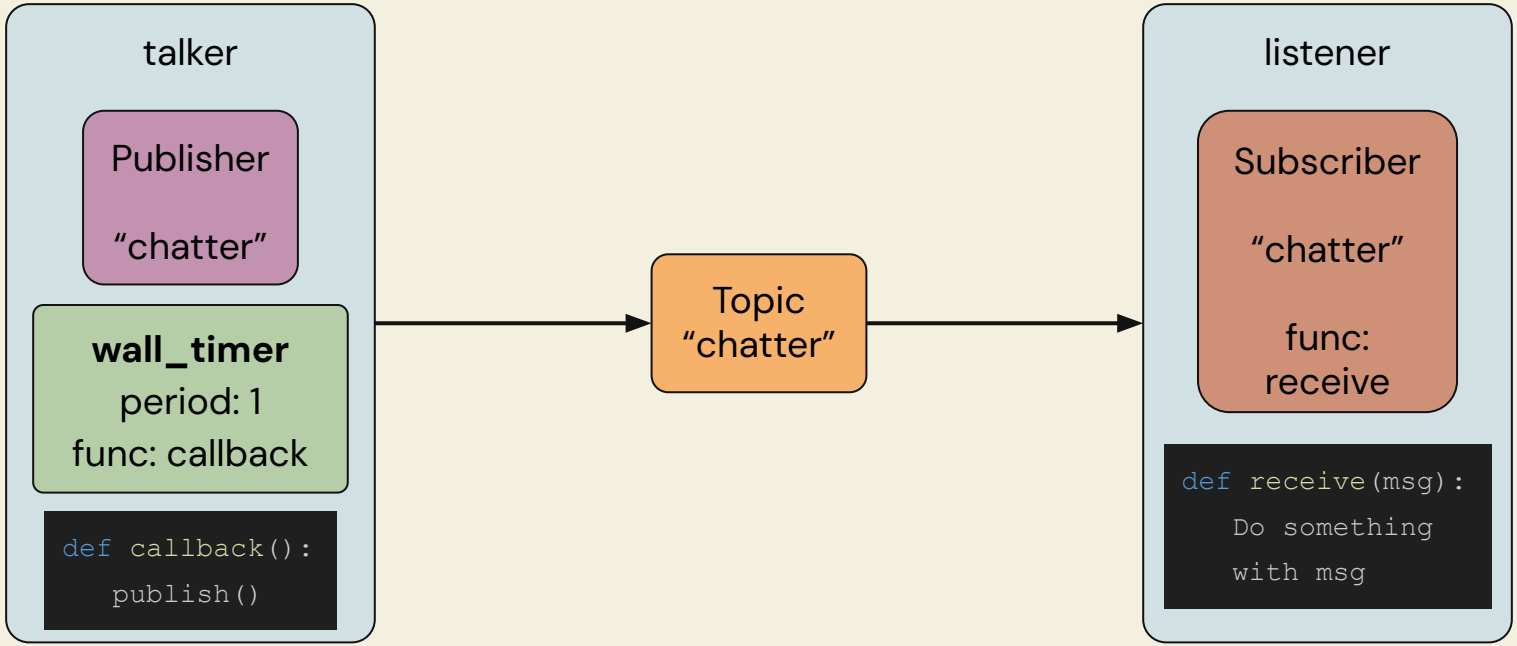

talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()
Topic
“chatter”
listener
Subscriber
“chatter”
wall_timer
period: 1
func: callback


talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()
Topic
“chatter”
listener
Subscriber
“chatter”
wall_timer

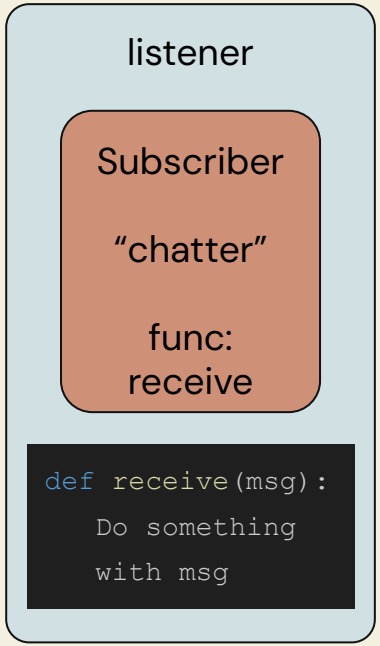
¿Porqué?


?
?
?
Topic
“chatter”
listener
Subscriber
“chatter”
wall_timer
period
func.
back

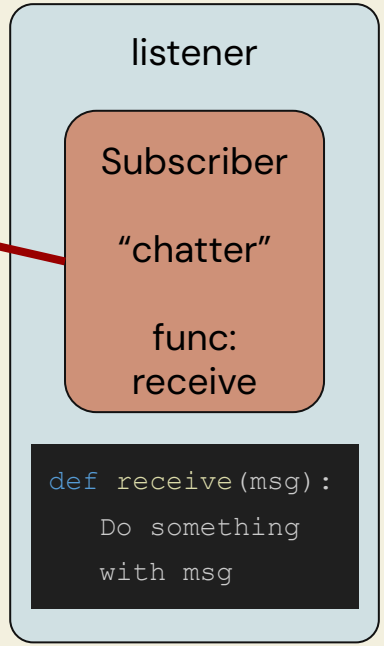
¿Porqué?


talker
Publisher
“chatter”
wall_timer
period: 1
func: callback
def callback():
publish()
Topic
“chatter”
listener
Subscriber
“chatter”
func:
receive
def receive(msg):
Do something
with msg

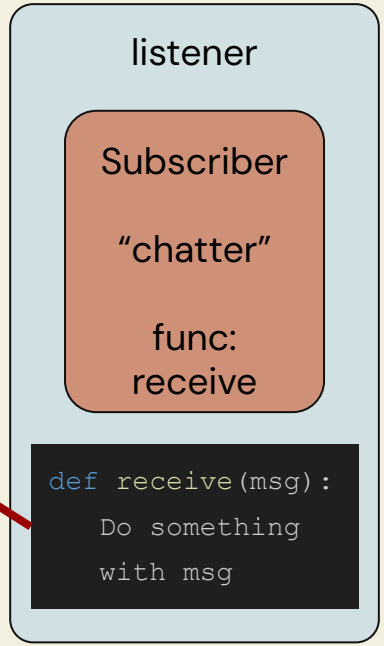
```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener')
        self.subscription = self.create_subscription(
            String,
            'chatter',
            self.callback,
            10  # QoS profile: queue size
        )
        self.subscription  # Prevent unused variable warning

    def callback(self, msg):
        self.get_logger().info(f"I heard: {msg.data}")

def main():
    rclpy.init()
    node = ListenerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener')
        self.subscription = self.create_subscription(
            String,
            'chatter',
            self.callback,
            10  # QoS profile: queue size
        )
        self.subscription  # Prevent unused variable warning

    def callback(self, msg):
        self.get_logger().info(f"I heard: {msg.data}")

def main():
    rclpy.init()
    node = ListenerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener')
        self.subscription = self.create_subscription(
            String,
            'chatter',
            self.callback,
            10  # QoS profile: queue size
        )
        self.subscription  # Prevent unused variable warning

    def callback(self, msg):
        self.get_logger().info(f"I heard: {msg.data}")

def main():
    rclpy.init()
    node = ListenerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```



Nuestro paquete

1. Modificamos setup.py → Agregamos el entry point del nuevo nodo

```
entry_points={
        'console_scripts': [
                'talker = pkg_name.my_file:main',
                'listener = pkg_name.my_other_file:main'
        ],
},
```

2. Compilamos todo desde la carpeta del workspace

```
colcon build --symlink-install
```

3. Hacemos “source install/setup.bash” (ojo si no tienen bash)

4. Corremos los nodos en terminales distintas

```
ros2 run pkg_name talker
ros2 run pkg_name listener
```

¿Qué pasa si solo corremos el nodo de “listener”?

¿El tópico existe?

Si corremos nuevamente el ejemplo de simulación del turtlebot, ¿qué tópicos veo?

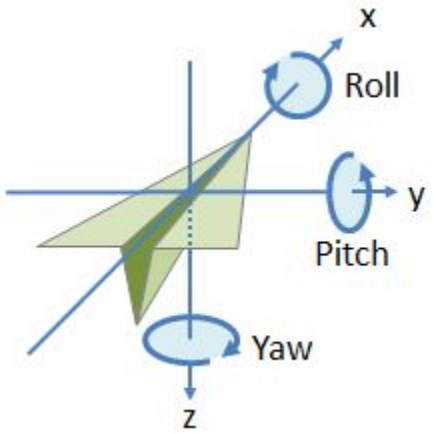
TP1

Cuaterniónes

Maneras de representar una orientación/giro en 3D

Maneras de representar una orientación/giro en 3D

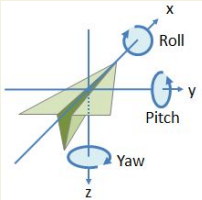
Ángulos de Euler (Roll, Pitch, Yaw)



Axis of Rotation	Euler Angle Name	Euler Angle Symbol
x	Roll	u
y	Pitch	v
z	Yaw	w

Maneras de representar una orientación/giro en 3D

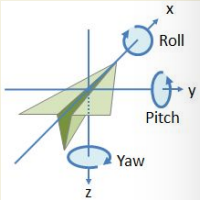
Ángulos de Euler (Roll, Pitch, Yaw)



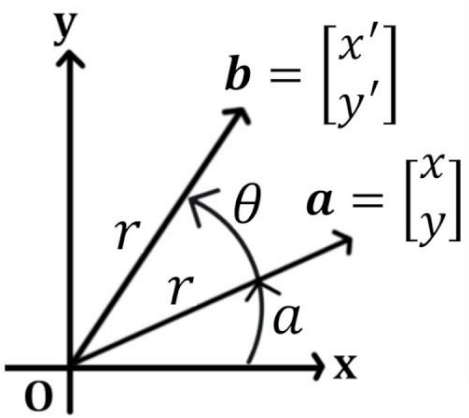
Axis of Rotation	Euler Angle Name	Euler Angle Symbol
x	Roll	u
y	Pitch	v
z	Yaw	w

Maneras de representar una orientación/giro en 3D

Angulos de
(Roll, Pitch



Rotation Matrix

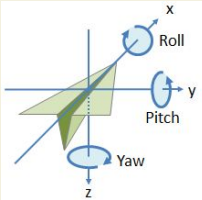


$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \begin{bmatrix} x \\ y \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

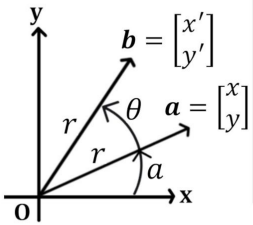
Maneras de representar una orientación/giro en 3D

Angulos de Euler (Roll, Pitch, Yaw)



Axis of Rotation	Euler Angle Name	Euler Angle Symbol
x	Roll	u
y	Pitch	v
z	Yaw	w

Rotation Matrix

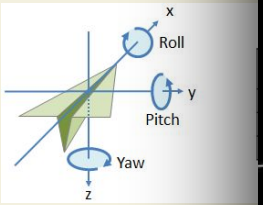


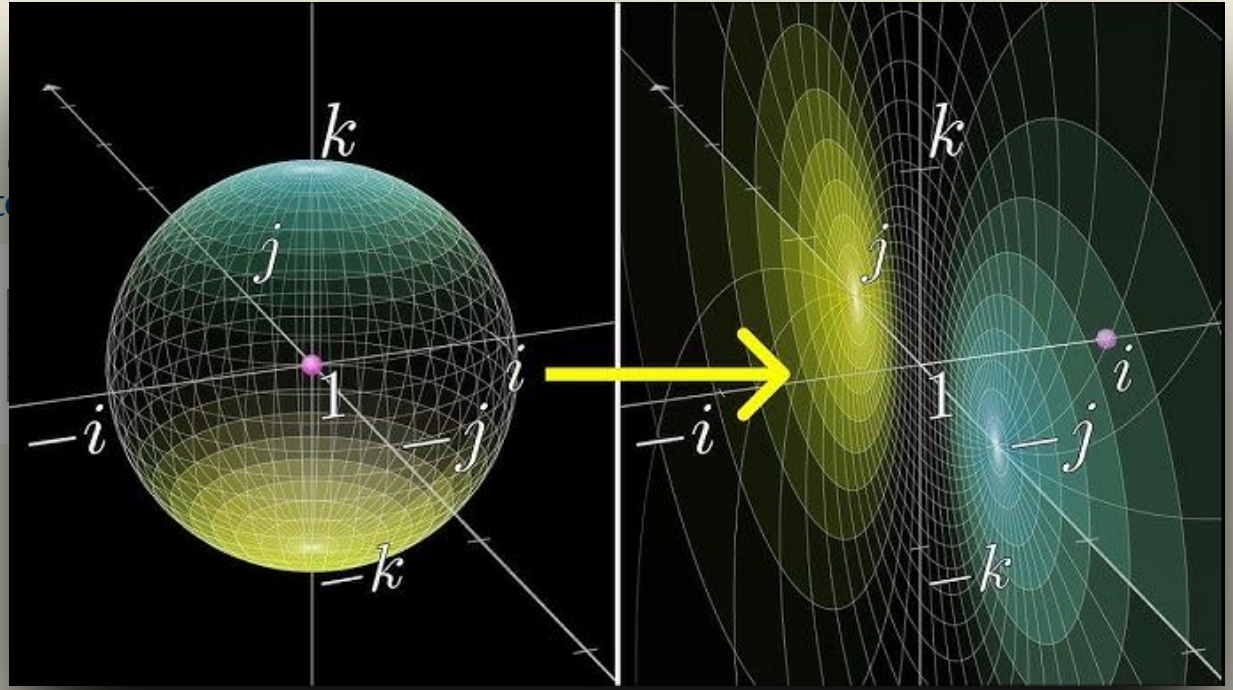
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \begin{bmatrix} x \\ y \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Maneras de representar una orientación/giro en 3D

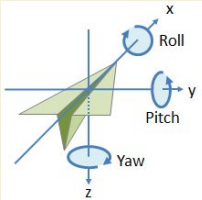
Angulos
(Roll, Pit





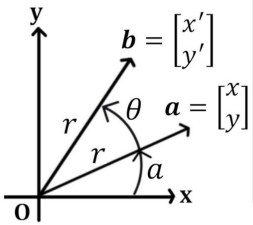
Maneras de representar una orientación/giro en 3D

Angulos de Euler (Roll, Pitch, Yaw)



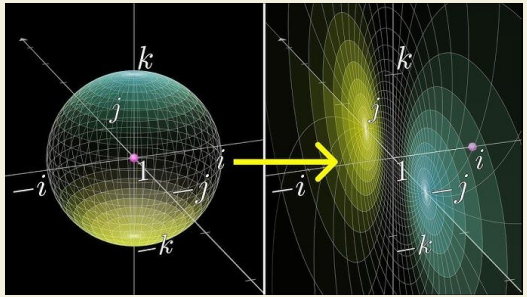
Axis of Rotation	Euler Angle Name	Euler Angle Symbol
x	Roll	u
y	Pitch	v
z	Yaw	w

Rotation Matrix

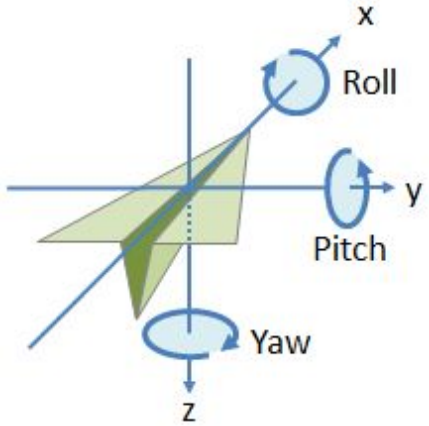


$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Cuaterniones



Ángulos de Euler (Roll, Pitch, Yaw)



Axis of Rotation	Euler Angle Name	Euler Angle Symbol
x	Roll	u
y	Pitch	v
z	Yaw	w

Ángulos de Euler (Roll, Pitch, Yaw)

PRO

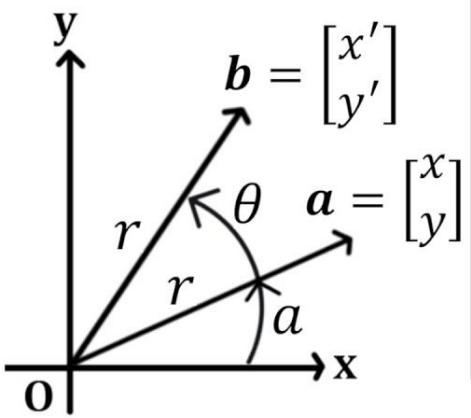
- Solo se necesitan 3 parámetros
- Intuitivo para humanos (nosotros)

CON

- Gimbal lock
  - Pierdo un grado de libertad
  - Singularidad
  - Dos sets de ángulos de euler pueden representar la misma orientación
- Hay que lidiar con lo cíclico de los ángulos
- Ecuaciones dinámicas de control sufren cerca de las singularidades
- Estimadores como EKF dependen de linealización y jacobianos que no son buenos con este método

Matriz de Rotación

Rotation Matrix



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \begin{bmatrix} x \\ y \end{bmatrix}$$

$$= \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Matriz de Rotación

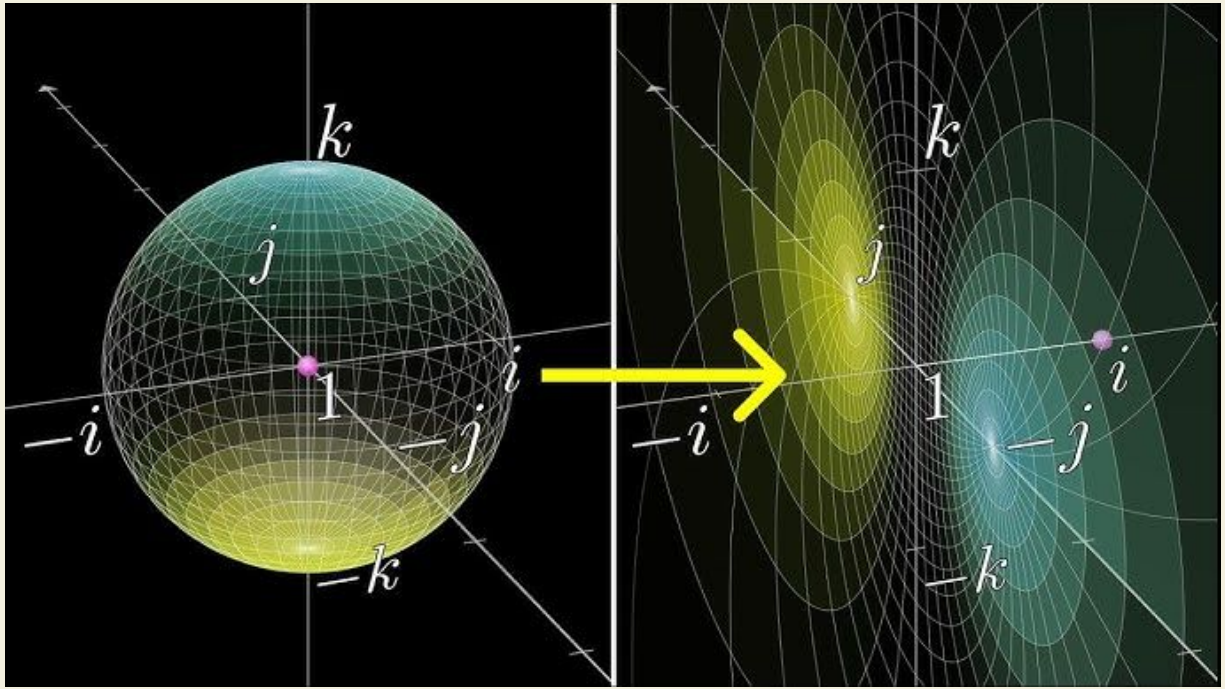
PRO

- No hay singularidades
- No hay Gimbal Lock
- No se pierde grado de libertad

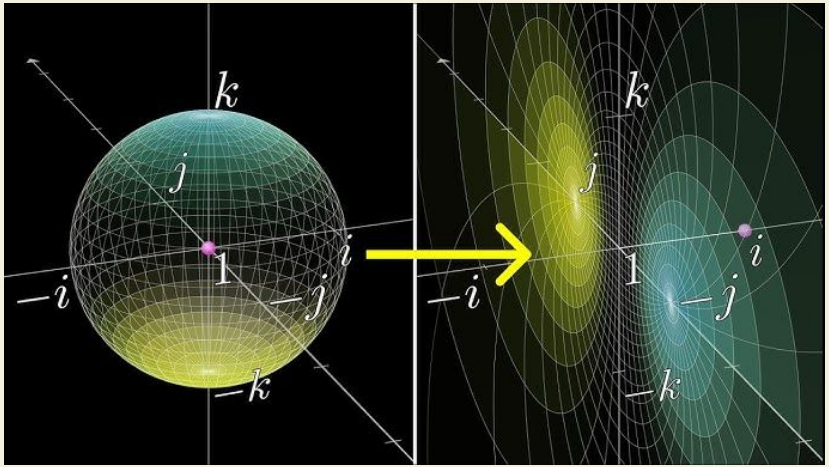
CON

- Estoy usando 9 números para representar 3 grados de libertad
- Requiere que la matriz tenga determinante 1 y que al multiplicar por la transpuesta de la identidad. Por errores numéricos suele empezar a driftear lejos de esto y se pierde esa cualidad
- Ineficiente y redundante para cálculos

Cuaterniones



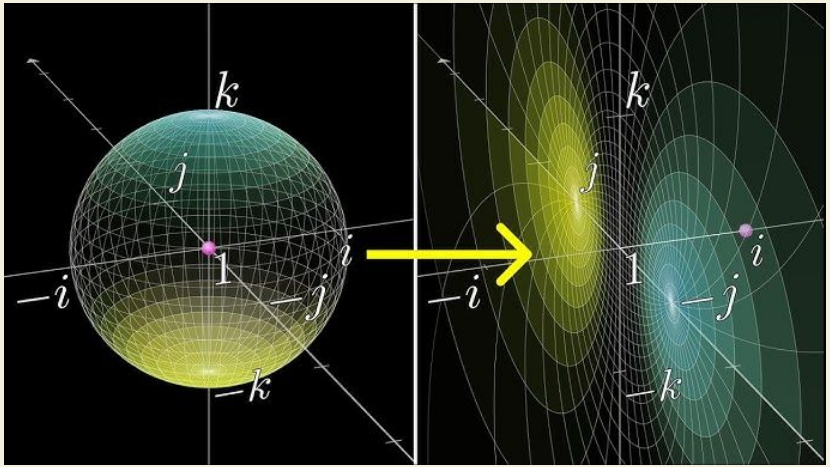
Cuaterniones



https://www.youtube.com/watch?v=d4EgbgTm0Bg

https://www.youtube.com/watch?v=zjMuIxRvygQ

Cuaterniones



Matriz de Rotación

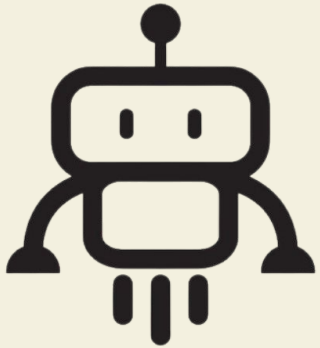
PRO

- No hay singularidades
- No hay Gimbal Lock
- No se pierden grados de libertad
- Buenos para algoritmos de control
  - Se puede calcular el error de manera suave y tiene buenos jacobianos
- Compacto
- Facil de linealizar
- Relación simple con velocidad angular
- Estable

CON

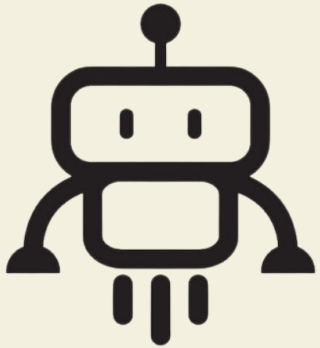
- Poco intuitivos
- Requiere normalizar para no perder cualidades

Workflow



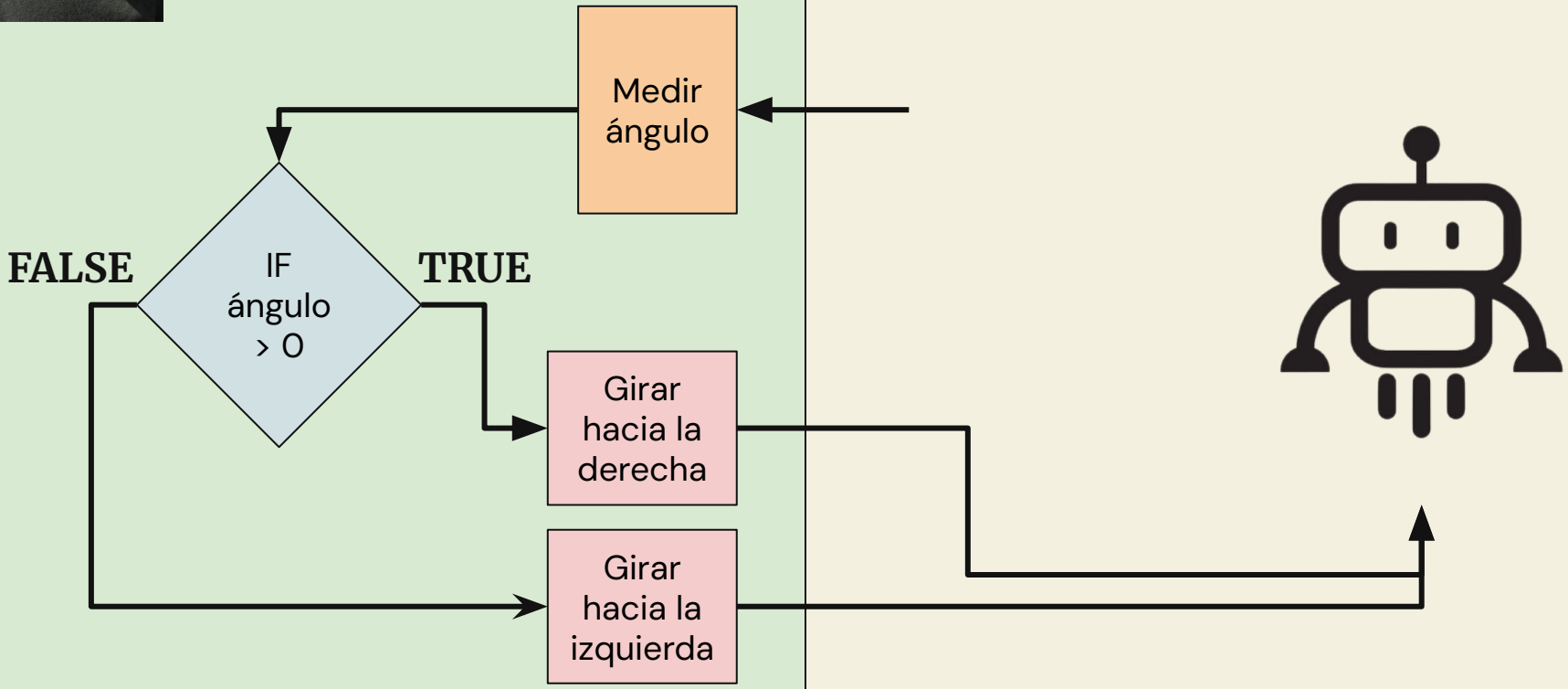
Workflow

Estoy controlando un robot 2D que solo tiene giro en z (YAW). Voy a hacer una lógica de control en yaw.



Ángulos de Euler (YAW)

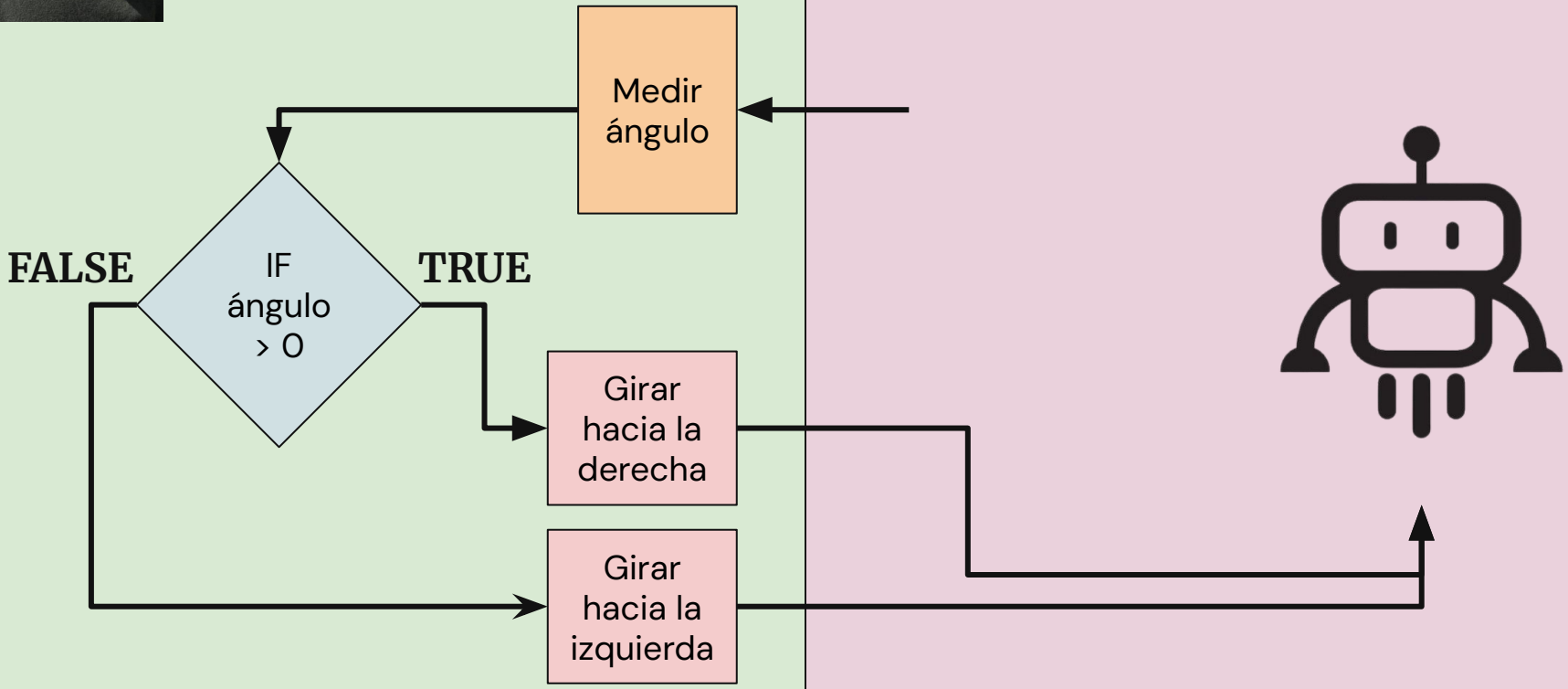
Workflow



Ángulos de Euler
(YAW)

Workflow

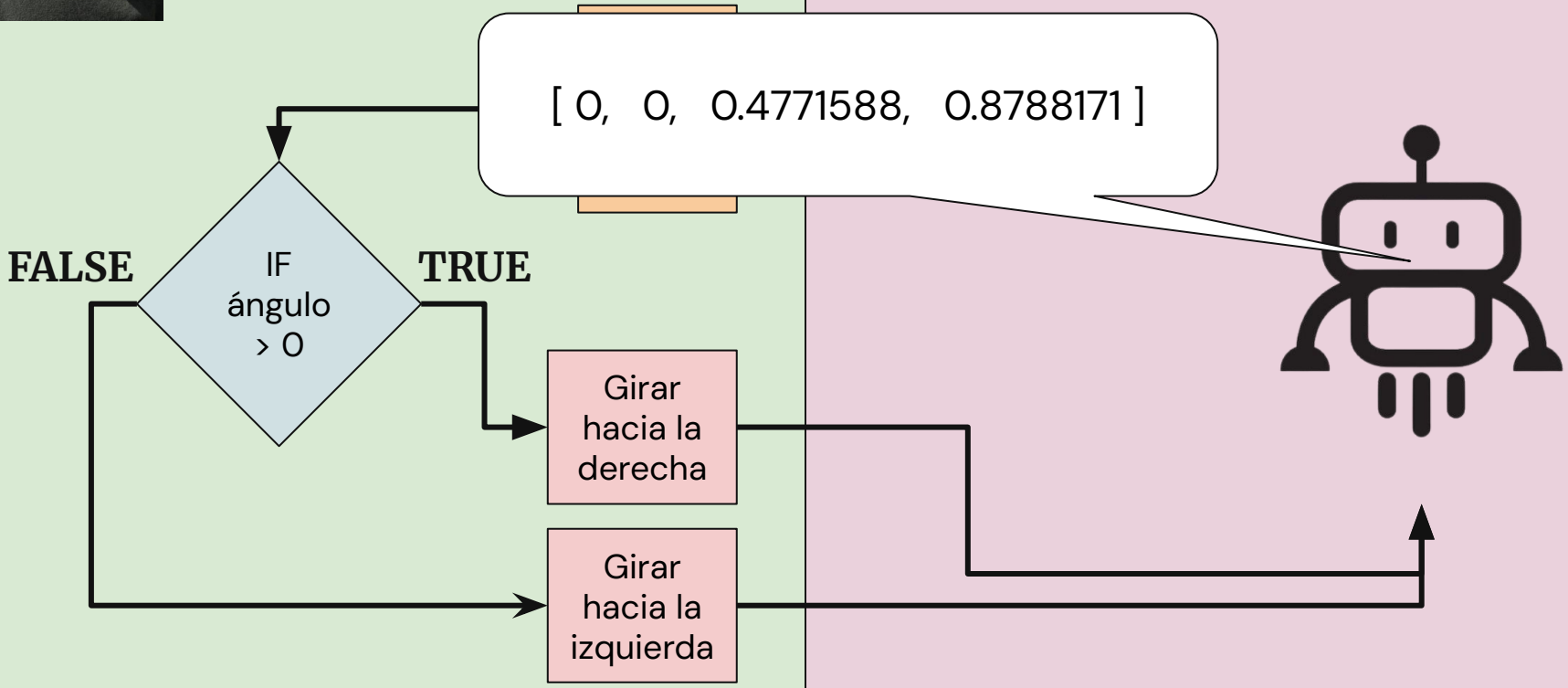
Cuaterniones



Ángulos de Euler
(YAW)

Workflow

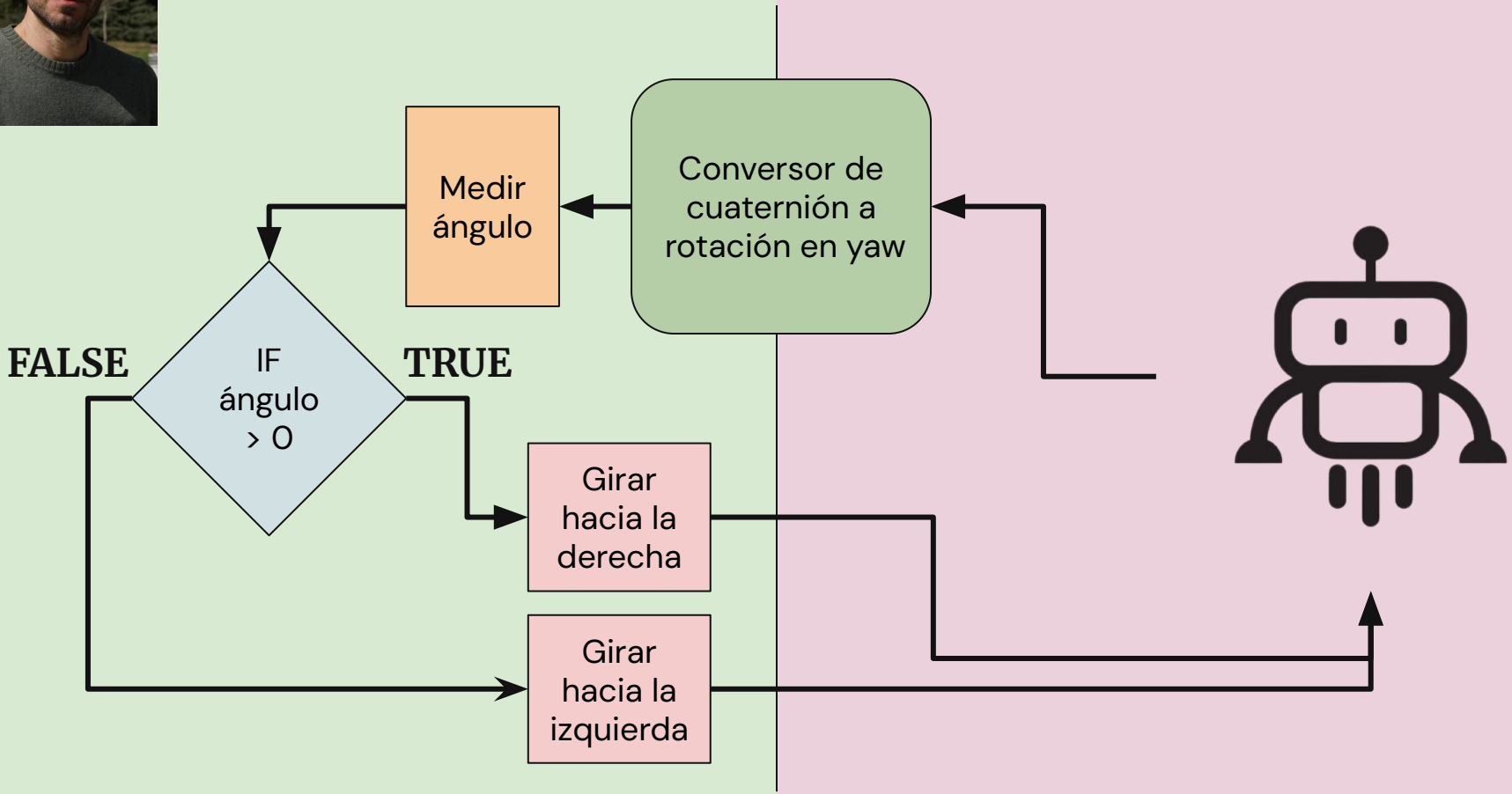
Cuaterniones



Ángulos de Euler
(YAW)

Workflow

Cuaterniones



TP1